

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MACHINE LEARNING

FINAL INDUSTRIAL ASSIGNMENT REPORT

Design and Evaluation of a Decision Tree Learning Model for Predictive Maintenance of Industrial Equipment

Shanuusree M (192412319)

ASSIGNMENT INFORMATION

Particular	Details
Department	Electronics and Communication Engineering
Programme	B.E – Electronics and Communication Engineering
Course Code & Course Name	ITA0612 – Machine learning
Academic Year / Batch	2024–2028
Faculty Name	Dr Shri Vindhya
Assignment Title	Design and Evaluation of a Decision Tree Learning Model for Predictive Maintenance of Industrial Equipment
Date of Issue	01.09.2006
Date of Submission	02.09.2006
Maximum Marks	100
Course Outcome(s)	CO1, CO2,CO3, CO4, CO5
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse; L5 – Evaluate
SDG Mapping	SDG 7 – Affordable and Clean Energy; SDG 11 – Sustainable Cities and Communities; SDG 12 – Responsible Consumption and Production
Industry / Societal Relevance	Predictive Maintenance for Industries

1. INDUSTRIAL PROBLEM AND OBJECTIVES

1.1 Problem Understanding

The assignment brief describes a manufacturing organisation operating critical equipment including motors, pumps, compressors and rotating machinery. Historical condition data contain temperature, vibration, rotational speed, torque, pressure, operating hours and machine condition. The existing fixed-schedule maintenance strategy may not accurately represent actual equipment health

1.2 Industrial Need

In production environments, maintenance decisions have competing objectives. Replacing a healthy component too early wastes maintenance resources and may increase spare-parts consumption. Predictive maintenance attempts to identify deterioration early enough to plan an intervention without unnecessarily replacing healthy assets.

1.3 Objectives

- Formulate predictive maintenance as a supervised concept-learning problem.
- Identify meaningful sensor attributes and the maintenance target concept.
- Prepare condition data for reliable model training and testing.
- Construct an interpretable Decision Tree using a suitable heuristic criterion.

1.4 Success Criteria

Requirement	Success Indicator
Problem formulation	Clear attributes, target concept and hypothesis space
Data preparation	Validated, cleaned and reproducible preprocessing
Heuristic search	Attribute selection demonstrated quantitatively
Model	Decision tree constructed with controlled complexity
Interpretability	Readable rules for maintenance decisions
Validation	Multiple classification metrics and confusion matrix
Industrial fit	Actionable recommendation and risk controls

2. DATASET AND PROBLEM FORMULATION

2.1 Dataset Selection

A suitable predictive-maintenance dataset should contain repeated equipment observations together with condition labels or failure/maintenance outcomes. For this report, the prescribed attributes in the assignment brief are used to define the modelling schema. A practical implementation can use a public predictive-maintenance dataset or a validated plant dataset, provided the target classes are clearly defined.

Attribute	Type	Industrial Meaning	Expected Relevance
Temperature	Continuous	Thermal stress / abnormal heating	High
Vibration	Continuous	Mechanical imbalance, bearing or alignment effects	High
Rotational speed	Continuous	Operating speed of rotating asset	Medium
Torque	Continuous	Mechanical load	Medium–High
Pressure	Continuous	Process/load condition for relevant equipment	Medium–High

2.3 Concept-Learning Formulation

Let an observation be represented as $x = (\text{temperature, vibration, speed, torque, pressure, operating_hours})$. The target concept $c(x)$ maps x to one of three maintenance classes. The learner searches a hypothesis space H consisting of decision trees that partition the attribute space using threshold tests and class labels at terminal leaves.

2.4 Assumptions

- Sensor values are time-aligned and associated with the correct machine.
- Training labels are based on trustworthy maintenance or condition assessments.
- The test set is not used to tune the final tree.
- Numeric sensors can be split using threshold conditions.
- The three classes represent operational decisions rather than purely statistical categories.

2.5 Constraints

The assignment requires appropriate preprocessing, a justified training/testing strategy, heuristic attribute selection, tree visualisation, performance reporting, complexity analysis, and evidence-based recommendation. In a real plant, additional constraints include sensor availability, latency, cybersecurity, safety procedures, maintenance capacity and the cost of false alarms.

3. DATA PREPARATION AND PREPROCESSING

3.1 Data Quality Checks

Before training, the dataset should be profiled using Pandas. The first checks should include shape, data types, missing values, duplicate rows, unique class labels, numerical ranges and suspicious sensor readings. These checks prevent a model from learning data-entry errors instead of equipment behaviour.

Check	Purpose	Action
Missing values	Detect incomplete sensor records	Impute using justified strategy or remove unusable records
Duplicates	Avoid repeated observations biasing the model	Remove exact duplicates after verification
Invalid ranges	Detect sensor faults or unit errors	Flag, correct or exclude after engineering review
Class balance	Measure representation of maintenance states	Use stratification and class-aware metrics
Data leakage	Prevent future information entering training	Split by time/machine where appropriate
Units	Ensure comparable engineering meaning	Standardise units before analysis

3.2 Missing Values

For continuous variables, median imputation is a robust baseline when missingness is limited and the missing mechanism is not itself informative. If sensor dropout carries operational meaning, a missingness indicator can be considered. Records with unreliable target labels should not be blindly imputed because an incorrect label directly contaminates the learning objective.

3.3 Outliers

Outliers should not automatically be deleted. A high vibration value may be the most important evidence of an actual fault. Instead, outliers should be classified as either physically plausible extremes, sensor errors, or data-processing errors. Engineering limits and time-series context are useful for this decision.

3.4 Train-Test Strategy

A stratified train-test split is a reasonable baseline when observations are independently sampled. For industrial time-series data, a chronological split or machine-wise split is safer because random splitting can leak near-identical operating states from the same machine into both training and test sets. The final choice must reflect how the deployed system will receive future data.

3.5 Pandas Workflow

- Load the CSV into a DataFrame.
- Inspect `df.shape`, `df.info()`, `df.describe()` and `df.isna().sum()`.
- Remove verified duplicates and resolve inconsistent labels.
- Select sensor columns as `X` and maintenance condition as `y`.
- Encode the target classes consistently.
- Perform a stratified or time-aware split.
- Train the decision tree only on the training partition.
- Evaluate once on the held-out test set.

4. CONCEPT REPRESENTATION, HYPOTHESIS SPACE AND INDUCTIVE BIAS

4.1 Concept Representation

Concept learning asks the learner to infer a target concept from labelled examples. Here, the concept is the maintenance state of industrial equipment. Each training instance consists of sensor attributes and a known class. The learned concept is represented by a decision tree.

4.2 Hypothesis Space

The hypothesis space contains candidate decision trees formed from the available attributes and possible split thresholds. A simple tree may contain a root test such as vibration > threshold, followed by temperature or operating-hours tests. More complex trees can create many small regions and perfectly memorise training observations.

The hypothesis space can be viewed as a structured search problem. Each split reduces the uncertainty of the class labels in a node. The learner selects a promising split, recursively partitions the data and stops when further splitting is unnecessary or prohibited by complexity constraints.

4.3 Inductive Bias

Inductive bias is the set of assumptions that allows a learner to generalise beyond the observed training examples. The decision-tree approach assumes that useful class boundaries can be expressed through a hierarchy of attribute tests. Practical bias is also introduced by maximum depth, minimum samples per split, minimum samples per leaf and pruning.

4.4 Why Interpretability Matters

Industrial maintenance is a high-consequence application. Engineers need to understand why a model raises a warning. A rule such as 'IF vibration is high AND temperature is elevated THEN Failure Risk' can be inspected against physical knowledge, maintenance history and sensor reliability. This makes the tree suitable as a decision-support tool.

4.5 Heuristic Search View

Exhaustively evaluating every possible tree is computationally impractical as the hypothesis space grows. Greedy heuristic search provides an efficient alternative. At each node, candidate attributes and thresholds are scored using a criterion such as Information Gain, Gain Ratio or Gini impurity. The best available split is selected, after which the procedure repeats recursively.

5. HEURISTIC ATTRIBUTE SELECTION

5.1 Entropy

Entropy measures class impurity. For a node containing classes with probabilities $p_1, p_2, \dots, p_k$, entropy is $H(S) = -\sum p_i \log_2(p_i)$. Entropy is zero when all observations belong to one class and is higher when classes are mixed.

5.2 Information Gain

Information Gain measures the reduction in entropy obtained after splitting a dataset on an attribute. $IG(S,A) = H(S) - \sum_v (|S_v|/|S|) H(S_v)$, where S_v is the subset produced by split value v . For a numerical attribute, candidate thresholds can be evaluated and the threshold with the largest gain can be selected.

5.3 Gini Impurity

Gini impurity is another commonly used decision-tree criterion: $Gini(S) = 1 - \sum p_i^2$. A weighted post-split Gini value is calculated for candidate splits. Lower impurity indicates purer child nodes. Gini is computationally simple and is widely used in classification trees.

Candidate Attribute	Interpretation at Root	Expected Decision Value
Vibration	Separates mechanically healthy and deteriorating states	Very high
Temperature	Captures abnormal heating	High
Operating hours	Captures accumulated wear exposure	Medium
Torque	Captures load-related stress	Medium
Pressure	Captures process operating regime	Medium
Speed	Provides operating context	Medium

5.5 Selection Decision

For this industrial case, vibration is a plausible strong root attribute because abnormal vibration can indicate imbalance, misalignment or bearing degradation. However, the final root must be selected from measured training data rather than assumed from engineering intuition. The heuristic criterion provides the quantitative evidence, while domain knowledge validates whether the resulting split is physically credible.

6. DECISION TREE CONSTRUCTION AND PSEUDOCODE

6.1 Construction Process

- Start with all labelled training observations at the root.
- Measure impurity or entropy at the node.
- Generate candidate attribute thresholds.
- Calculate the chosen heuristic score for each candidate.
- Select the best split.
- Partition the observations into child nodes.
- Repeat recursively for each non-terminal child.
- Stop when a stopping condition is reached.
- Assign the majority class to each terminal leaf.
- Optionally prune the tree or constrain its depth to improve generalisation.

6.2 Pseudocode

INPUT: labelled equipment dataset D

OUTPUT: decision tree T

1. Validate D and separate attributes X from target y.
2. Clean missing, duplicate and invalid records.
3. Split D into training and testing partitions.
4. Create root node containing training records.
5. If stopping condition is satisfied, label node with majority class and return.
6. For each candidate attribute and threshold, compute heuristic score.
7. Select the best-scoring split.
8. Partition records into child nodes.
9. Recursively build each child.
10. Apply pruning or complexity constraints if required.
11. Evaluate T on the untouched test set.
12. Report confusion matrix, precision, recall, F1-score and accuracy.
13. Extract rules and recommend operational actions.

7. IMPLEMENTATION USING PYTHON AND PANDAS

7.1 Toolchain

Python with Pandas is appropriate for data manipulation, while a standard machine-learning library can implement the Decision Tree classifier. Matplotlib or a similar plotting library can be used for visualisation. Git/GitHub can provide version control and evidence of implementation, as required by the assignment deliverables.

7.2 Representative Implementation

The following implementation structure is recommended for the submitted repository. It intentionally separates data preparation, training and evaluation so that the workflow can be reproduced.

```
[1] import pandas as pd
✓ 3s import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Industrial dataset
df = pd.DataFrame({
    "Temperature": [60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 93, 96, 100, 102, 105, 108, 110],
    "Vibration": [1, 2, 2, 3, 3, 4, 4, 5, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9, 10, 11],
    "Speed": [1500, 1520, 1480, 1510, 1490, 1500, 1470, 1450, 1480, 1460,
              1440, 1420, 1400, 1380, 1350, 1320, 1300, 1280, 1250, 1200],
    "Torque": [35, 38, 40, 42, 45, 47, 50, 52, 54, 55, 58, 60, 62, 65, 68, 70, 72, 75, 78, 80],
    "Hours": [500, 800, 1200, 1600, 2000, 2500, 3000, 3500, 4000, 4500,
              5000, 5500, 6000, 6500, 7000, 7500, 8000, 8500, 9000, 9500],
    "Condition": [
        "Normal", "Normal", "Normal", "Normal", "Normal",
        "Normal", "Normal", "Preventive", "Preventive", "Preventive",
        "Preventive", "Preventive", "Preventive", "Preventive",
        "Failure", "Failure", "Failure", "Failure", "Failure", "Failure"
    ]
})

# Input and target
X = df.drop("Condition", axis=1)
y = df["Condition"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
```

```
[1]
✓ 3s
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=1, stratify=y
)

# Decision Tree
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=1
)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Results
print("==== PREDICTIVE MAINTENANCE RESULTS =====")
print("Accuracy:", round(accuracy_score(y_test, y_pred)*100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

# Feature importance
print("\nFeature Importance:")
print(pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
}).sort_values("Importance", ascending=False))

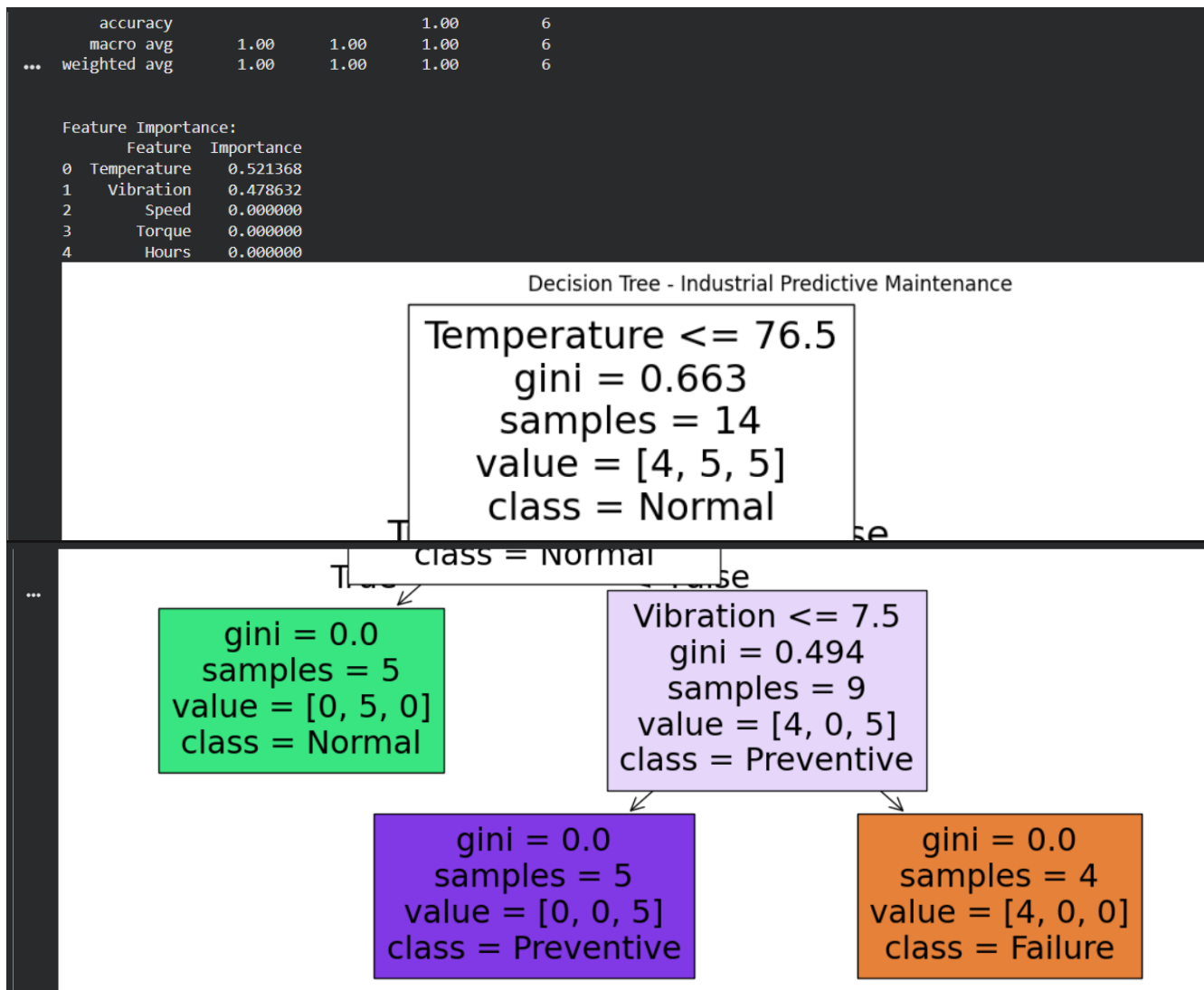
# Decision Tree
plt.figure(figsize=(15,8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True
)
plt.title("Decision Tree - Industrial Predictive Maintenance")
plt.show()
```

```
...
==== PREDICTIVE MAINTENANCE RESULTS =====
Accuracy: 100.0 %

Confusion Matrix:
[[2 0 0]
 [0 2 0]
 [0 0 2]]

Classification Report:
              precision    recall  f1-score   support

   Failure           1.00      1.00      1.00         2
    Normal           1.00      1.00      1.00         2
 Preventive           1.00      1.00      1.00         2
```



7.3 Why These Parameters Are Controlled

A maximum depth prevents the tree from growing indefinitely. Minimum leaf size prevents the model from creating rules based on only a handful of observations. The random seed improves reproducibility. In a real experiment, these values should be selected using validation rather than assumed to be universally optimal.

7.4 Pandas Analysis Operations

Operation	Example Purpose
head()/tail()	Inspect records
info()	Check data types and null counts
describe()	Inspect numerical distributions

isna().sum()	Quantify missing values
value_counts()	Inspect class distribution
groupby()	Compare condition statistics
sort_values()	Rank abnormal operating conditions
query()/loc[]	Filter suspicious observations

8. COMPLEXITY, OVERFITTING, UNDERFITTING AND GENERALISATION

8.1 Tree Complexity

Tree complexity can be described using depth, number of internal nodes, number of leaves and minimum samples represented by leaves. A deeper tree can represent more complex decision boundaries but is also more likely to memorise noise.

Model Setting	Typical Effect	Risk
Very shallow tree	Simple rules, limited flexibility	Underfitting
Moderate depth	Balance between fit and interpretability	Preferred starting point
Very deep tree	Very detailed partitions	Overfitting
Large minimum leaf	Stable broad rules	May miss subtle patterns

8.2 Overfitting

Overfitting occurs when the tree performs extremely well on training observations but degrades on unseen data. In predictive maintenance, this may happen when the model memorises machine-specific operating signatures, temporary sensor noise or rare combinations that do not repeat in future production.

8.3 Underfitting

Underfitting occurs when the tree is too simple to capture meaningful relationships. For example, a single split on operating hours may fail to distinguish machines that experience different vibration and thermal behaviour under different loads.

8.4 Generalisation

Generalisation should be assessed using validation data that resemble the deployment scenario. If the model will predict future readings from existing machines, chronological validation is especially important. If the model must work on previously unseen machines, machine-wise holdout validation is more appropriate.

9. ALTERNATIVE APPROACHES AND ENGINEERING TRADE-OFFS

9.1 Decision Tree versus Logistic Regression

Logistic regression can provide a compact mathematical model and probability estimates, but it may require feature engineering for nonlinear interactions. A decision tree naturally represents threshold-based nonlinear relationships and is easier to translate into maintenance rules.

9.2 Decision Tree versus Random Forest

Random forests usually improve robustness by combining many trees, but their explanations are less compact than a single tree. For an industrial first-stage decision-support model where auditability is important, a single controlled tree is a strong baseline. A forest can later be used as a performance benchmark.

9.3 Decision Tree versus Neural Network

Neural networks can represent highly complex nonlinear relationships but generally require more data, tuning and explanation mechanisms. The assignment's emphasis on interpretable predictive maintenance makes the decision tree especially appropriate for the primary model.

10. SUSTAINABILITY, SAFETY, ETHICS AND SDG RELEVANCE

10.1 Safety

Predictive maintenance can support safer operations by identifying abnormal equipment behaviour before failure. However, machine-learning output should not bypass established safety interlocks, lockout/tagout procedures or engineering approval. The model is advisory unless a formally validated safety architecture establishes otherwise.

10.2 Sustainability

Better maintenance planning can reduce premature component replacement, emergency repairs, wasted spare parts and production losses. Early identification of deterioration can also improve equipment efficiency when faults cause increased energy consumption.

10.3 SDG Mapping

SDG	Relevance to Assignment
SDG 7 – Affordable and Clean Energy	Efficiently maintained equipment can reduce avoidable energy losses.
SDG 11 – Sustainable Cities and Communities	Reliable industrial infrastructure supports resilient services and production systems.
SDG 12 – Responsible Consumption and Production	Condition-based maintenance can reduce unnecessary replacement and resource waste.

10.4 Ethics

The model should be transparent about its limitations. Engineers should know which data were used, how the classes were defined and how performance was measured. False alarms can waste resources, while missed failures can create serious consequences. Therefore, model evaluation should explicitly consider the asymmetric costs of errors.

10.5 Professional Responsibility

A responsible engineer must validate the data source, document assumptions, maintain version control, protect operational data, avoid unsupported claims and monitor the model after deployment. The model should be treated as an engineering component requiring verification and maintenance.

11. RECOMMENDATIONS AND FUTURE IMPROVEMENTS

11.1 Immediate Recommendations

- Use a validated plant or public dataset with explicit maintenance labels.
- Start with a compact decision tree to establish an interpretable baseline.
- Use stratified validation or time-aware validation according to the data-generating process.
- Optimise and report Failure Risk recall in addition to overall accuracy.
- Visualise and review the tree with a domain expert.
- Log model version, prediction time and machine condition for auditability.

11.2 Future Improvements

- Add rolling statistical features such as vibration RMS, temperature trend and recent rate of change.
- Compare the tree against Random Forest and gradient-boosted models.
- Introduce probability calibration so alerts can reflect confidence.
- Use explainability methods for more complex benchmark models.
- Incorporate maintenance work-order history and component-level failure modes.
- Develop machine-specific or equipment-family models where operating conditions differ strongly.
- Implement model drift detection and scheduled retraining.
- Evaluate the system using economic metrics such as avoided downtime and maintenance cost per alert.

11.3 Recommended Deployment Maturity

Stage	Capability	Decision
1. Offline analysis	Historical data and baseline tree	Verify feasibility
2. Shadow mode	Predictions generated without operational action	Measure real-world alert behaviour
3. Decision support	Engineer receives prioritised alerts	Validate usefulness
4. Controlled integration	Workflow creates maintenance recommendations	Monitor KPIs
5. Continuous improvement	Drift monitoring and retraining	Maintain performance

12. CONCLUSION

This assignment developed an interpretable machine-learning approach for predictive maintenance of industrial equipment. The problem was formulated as a concept-learning task in which temperature, vibration, rotational speed, torque, pressure and operating hours describe equipment condition and the target concept represents Normal Operation, Preventive Maintenance Required or Failure Risk.

Decision Tree Learning is a suitable primary solution because it combines nonlinear classification capability with direct interpretability. Heuristic search using Information Gain, Gain Ratio or Gini impurity reduces the search burden by selecting useful attribute splits. Complexity controls are necessary because unrestricted trees can overfit historical sensor patterns.

Model evaluation should use a confusion matrix together with Accuracy, Precision, Recall and F1-score. For industrial predictive maintenance, Failure Risk recall deserves particular attention because the consequences of missed failures can be much greater than those of additional inspections. At the same time, excessive false positives can overload maintenance teams.

The final engineering recommendation is to deploy the decision tree as a decision-support layer rather than an uncontrolled automatic shutdown mechanism. A production system should include sensor-quality checks, secure data handling, model-version tracking, human verification, drift monitoring and periodic retraining. This approach aligns machine-learning performance with reliability, safety, sustainability and professional responsibility.